

Namespace ExBrainSdk.HighLevel

Classes

[Block](#)

全フレームデータを含む完了済み計測ブロック / A completed measurement block with all frame data.

[CsvExtensions](#)

CSV 出力用拡張メソッド群 / Extension methods for CSV export.

[Experiment](#)

ブロック進行とデータ収集を管理するハイレベル実験 API。所有者は API / High-level experiment API that manages block sequencing and data collection. Owned by the API.

[FrameData](#)

完了した Block に格納される単一の処理済み計測フレーム / A single processed measurement frame stored in a completed Block.

[InstantFrame](#)

SMD 値を含むハイレベルのリアルタイム計測フレーム / High-level real-time measurement frame including SMD values.

[JsonExtensions](#)

JSON シリアライズ拡張メソッド群 / Extension methods for JSON serialization.

[MetaData](#)

計測ブロックの時間フェーズ設定。所有者は API / Configuration for a measurement block's timing phases. Owned by the API.

[TaskComparison](#)

2 つのブロック集合間の統計比較結果。インスタンス生成は static メソッドを使用 / Result of a statistical comparison between two sets of blocks. Use the static methods to create instances.

Enums

[BlockStage](#)

計測ブロックの 4 つのフェーズ / The four phases of a measurement block.

[StatisticalTest](#)

TaskComparison で使用される統計検定種別 / Statistical test used in a TaskComparison.

[Tail](#)

統計検定の片側・両側指定 / Tail direction for statistical tests.

Delegates

[Experiment.NotifyInstantFrameCallback](#)

SMD 値付きリアルタイム計測フレームごとに呼び出されるコールバック / Callback invoked for each real-time measurement frame with SMD values.

[Experiment.NotifyStageStartedCallback](#)

ブロックの各フェーズ開始時に呼び出されるコールバック / Callback invoked when each phase of a block begins.

Class Block

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

全フレームデータを含む完了済み計測ブロック / A completed measurement block with all frame data.

```
public sealed class Block
```

Inheritance

[object](#)  ← Block

Extension Methods

[CsvExtensions.ToCsv\(Block\)](#), [JsonExtensions.ToJson\(Block\)](#)

Constructors

Block()

JSON 逆シリアライズまたは手動構築のために空の Block を作成する / Creates an empty Block for JSON deserialization or manual construction.

```
public Block()
```

Properties

AverageL1

ブロック作成時に取得した平均値 (L1cm) / Average (L1cm) captured at block creation time

```
public int AverageL1 { get; set; }
```

Property Value

[int](#) 

AverageL3

ブロック作成時に取得した平均値 (L3cm) / Average (L3cm) captured at block creation time

```
public int AverageL3 { get; set; }
```

Property Value

[int](#)

AverageR1

ブロック作成時に取得した平均値 (R1cm) / Average (R1cm) captured at block creation time

```
public int AverageR1 { get; set; }
```

Property Value

[int](#)

AverageR3

ブロック作成時に取得した平均値 (R3cm) / Average (R3cm) captured at block creation time

```
public int AverageR3 { get; set; }
```

Property Value

[int](#)

CalmDownFrames

クールダウンフェーズのフレーム群 / Frames from the calm-down phase

```
public Collection<FrameData> CalmDownFrames { get; set; }
```

Property Value

Collection<[FrameData](#)>

DeviceId

ブロック作成時に取得した装置 ID / Device ID captured at block creation time

```
public string DeviceId { get; set; }
```

Property Value

[string](#)

GainL1

ブロック作成時に取得したゲイン (L1cm) / Gain (L1cm) captured at block creation time

```
public int GainL1 { get; set; }
```

Property Value

[int](#)

GainL3

ブロック作成時に取得したゲイン (L3cm) / Gain (L3cm) captured at block creation time

```
public int GainL3 { get; set; }
```

Property Value

[int](#)

GainR1

ブロック作成時に取得したゲイン (R1cm) / Gain (R1cm) captured at block creation time

```
public int GainR1 { get; set; }
```

Property Value

[int](#)

GainR3

ブロック作成時に取得したゲイン (R3cm) / Gain (R3cm) captured at block creation time

```
public int GainR3 { get; set; }
```

Property Value

[int](#)

Identity

参加者またはセッションの識別子 / Participant or session identity

```
public string Identity { get; set; }
```

Property Value

[string](#)

Name

ブロック名 / Block name

```
public string Name { get; set; }
```

Property Value

[string](#)

PostFittingFrames

事後装着調整フェーズのフレーム群 / Frames from the post-fitting phase

```
public Collection<FrameData> PostFittingFrames { get; set; }
```

Property Value

Collection<[FrameData](#)>

PreFittingFrames

事前装着調整フェーズのフレーム群 / Frames from the pre-fitting phase

```
public Collection<FrameData> PreFittingFrames { get; set; }
```

Property Value

Collection<[FrameData](#)>

RepeatIndex

実験内での同一ブロック名に対する 0 始まりの繰り返し番号 / Zero-based repeat count for this block name within the experiment

```
public int RepeatIndex { get; set; }
```

Property Value

[int](#)

TaskFrames

タスクフェーズのフレーム群 / Frames from the task phase

```
public Collection<FrameData> TaskFrames { get; set; }
```

Property Value

Collection<[FrameData](#)>

Methods

FromJson(ExBrainApi, string)

JSON 文字列を Block に逆シリアライズし、ネイティブ API へ登録する / Deserializes a JSON string into a Block and registers it with the native API.

```
public static Block? FromJson(ExBrainApi api, string json)
```

Parameters

api [ExBrainApi](#)

ネイティブブロックを保持する API インスタンス / The API instance that will own the native block.

json [string](#) 

[ToJson\(Block\)](#) で生成された JSON 文字列 / JSON string produced by [ToJson\(Block\)](#).

Returns

[Block](#)

逆シリアライズされた Block インスタンス / The deserialized Block instance.

Exceptions

Exception

JSON の逆シリアライズまたはネイティブブロック生成に失敗した場合 / Thrown if deserialization fails or native block creation fails.

Enum BlockStage

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

計測ブロックの 4 つのフェーズ / The four phases of a measurement block.

```
public enum BlockStage
```

Fields

CalmDown = 2

クールダウンフェーズ / Calm-down phase.

PostFitting = 3

事後装着調整フェーズ / Post-fitting phase.

PreFitting = 0

事前装着調整フェーズ / Pre-fitting phase.

Task = 1

タスクフェーズ / Task phase.

Class CsvExtensions

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

CSV 出力用拡張メソッド群 / Extension methods for CSV export.

```
public static class CsvExtensions
```

Inheritance

[object](#)  ← CsvExtensions

Methods

ToCsv(Block)

[Block](#) を 4 フェーズすべてを含む CSV 文字列へ変換する / Converts a [Block](#) to a CSV string containing all four stages.

```
public static string ToCsv(this Block block)
```

Parameters

block [Block](#)

変換対象ブロック / The block to convert.

Returns

[string](#) 

ブロックの CSV 文字列表現 / A CSV string representation of the block.

Remarks

形式は low-level C++ MeasurementLogger の CSV と同等: The format mirrors the low-level C++ MeasurementLogger CSV:

- **#** で始まるコメント行 / Comment lines prefixed with **#**

- カラムヘッダ行 / A column-header row
- PreFitting, Task, CalmDown, PostFitting の各フレームごとのデータ行 / One data row per frame across PreFitting, Task, CalmDown, and PostFitting stages

Class Experiment

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

ブロック進行とデータ収集を管理するハイレベル実験 API。所有者は API / High-level experiment API that manages block sequencing and data collection. Owned by the API.

```
public sealed class Experiment
```

Inheritance

[object](#)  ← Experiment

Constructors

Experiment(ExBrainApi, int, NotifyInstantFrameCallback?)

Experiment インスタンスを生成する / Creates an Experiment instance.

```
public Experiment(ExBrainApi api, int smdTargetWindowSize,  
Experiment.NotifyInstantFrameCallback? onMeasurement = null)
```

Parameters

api [ExBrainApi](#)

この Experiment を保持する初期化済み [ExBrainApi](#) インスタンス / Initialized [ExBrainApi](#) instance that will own this Experiment.

smdTargetWindowSize [int](#) 

SMD ターゲットウィンドウに使用する最新フレーム数（1 以上） / Number of most recent frames used as the SMD target window (must be positive).

onMeasurement [Experiment.NotifyInstantFrameCallback](#)

計測フレーム受信時に呼ばれる任意コールバック / Optional callback invoked for each measurement frame with SMD values.

Exceptions

InvalidOperationException

ネイティブ Experiment の作成に失敗した場合 / Thrown when native experiment creation fails.

Methods

FinishBlockAsync(NotifyStageStartedCallback?)

現在のブロックを終了する（クールダウンと事後装着調整フェーズを実行）。後続フェーズ完了時に終了し、戻り値の [Block](#) はこの [Experiment](#) の寿命中有効 / Finishes the current block (runs calm-down and post-fitting phases). Completes when all post-block phases finish. The returned [Block](#) is valid for the lifetime of this [Experiment](#).

```
public Task<Block> FinishBlockAsync(Experiment.NotifyStageStartedCallback? onStageStarted  
= null)
```

Parameters

onStageStarted [Experiment.NotifyStageStartedCallback](#)

各フェーズ開始時に呼ばれる任意コールバック（CalmDown と PostFitting を通知） / Optional callback invoked when each phase begins (fires for CalmDown and PostFitting).

Returns

[Task](#) [<Block>](#)

完了したブロックを返すタスク / A task that returns the completed block.

FinishExperimentAsync()

計測を停止し、CSV ロガーをフラッシュする。装置が計測停止を通知した時点で完了する / Stops measurement and flushes the CSV logger. Completes when the device confirms measurement has stopped.

```
public Task<EnmSdkResult> FinishExperimentAsync()
```

Returns

[Task](#) [<EnmSdkResult>](#)

停止処理の結果コードを返すタスク / A task that returns the stop result code.

FinishReferencePeriod()

現在開いている参照期間の終了を記録する / Marks the end of the currently open reference period.

```
public EnmSdkResult FinishReferencePeriod()
```

Returns

[EnmSdkResult](#)

結果コード / Result code.

RunBlockAsync(MetaData, string, string, NotifyStageStartedCallback?)

完全なブロック（事前装着調整、タスク、クールダウン、事後装着調整）を自動実行する。全フェーズ完了時に終了し、戻り値の [Block](#) はこの [Experiment](#) の寿命中有効 / Runs a complete block (pre-fitting, task, calm-down, post-fitting) unattended. Completes when all phases finish. The returned [Block](#) is valid for the lifetime of this [Experiment](#).

```
public Task<Block> RunBlockAsync(MetaData metaData, string blockName, string identity,
    Experiment.NotifyStageStartedCallback? onStageStarted = null)
```

Parameters

metaData [MetaData](#)

ブロック進行設定 / Block progression metadata.

blockName [string](#)

ブロック名 / Block name.

identity [string](#)

参加者またはセッション識別子 / Participant or session identity.

onStageStarted [Experiment.NotifyStageStartedCallback](#)

各フェーズ開始時に呼ばれる任意コールバック（PreFitting / Task / CalmDown / PostFitting の順に通知） / Optional callback invoked when each phase begins (fires for PreFitting, Task, CalmDown, PostFitting in order).

Returns

[Task](#)  <[Block](#)>

完了したブロックを返すタスク / A task that returns the completed block.

SetBandpassFilter(int, double, double, double)

対象フィールドへ適用する Butterworth バンドパスフィルタを設定する（1回のみ設定可） / Sets a Butterworth bandpass filter applied to interested fields. Can only be set once.

```
public EnmSdkResult SetBandpassFilter(int order, double lowCutoff, double highCutoff,
double sampleRate)
```

Parameters

order [int](#) 

フィルタ次数 / Filter order.

lowCutoff [double](#) 

下限カットオフ周波数 / Low cutoff frequency.

highCutoff [double](#) 

上限カットオフ周波数 / High cutoff frequency.

sampleRate [double](#) 

サンプリング周波数 / Sampling rate.

Returns

[EnmSdkResult](#)

結果コード / Result code.

SetLowpassFilter(int, double, double)

対象フィールドへ適用する Butterworth ローパスフィルタを設定する（1回のみ設定可） / Sets a Butterworth lowpass filter applied to interested fields. Can only be set once.

```
public EnmSdkResult SetLowpassFilter(int order, double cutOff, double sampleRate)
```

Parameters

order [int](#)

フィルタ次数 / Filter order.

cutOff [double](#)

カットオフ周波数 / Cutoff frequency.

sampleRate [double](#)

サンプリング周波数 / Sampling rate.

Returns

[EnmSdkResult](#)

結果コード / Result code.

StartBlockAsync(MetaData, string, string, NotifyStageStartedCallback?)

ブロックを開始する（事前装着調整実行後、[FinishBlockAsync\(NotifyStageStartedCallback?\)](#) で終了） / Starts a block (runs pre-fitting phase, then awaits [FinishBlockAsync\(NotifyStageStartedCallback?\)](#)). Completes when the pre-fitting phase finishes and the task phase begins.

```
public Task<EnmSdkResult> StartBlockAsync(MetaData metaData, string blockName, string identity, Experiment.NotifyStageStartedCallback? onStageStarted = null)
```


Parameters

metaData [MetaData](#)

ブロック進行設定 / Block progression metadata.

blockName [string](#)

ブロック名 / Block name.

identity [string](#)

参加者またはセッション識別子 / Participant or session identity.

onStageStarted [Experiment.NotifyStageStartedCallback](#)

各フェーズ開始時に呼ばれる任意コールバック（StartBlockAsync では PreFitting と Task が通知される） /
Optional callback invoked when each phase begins (PreFitting and Task fire for StartBlockAsync).

Returns

[Task](#) <[EnmSdkResult](#)>

開始処理の結果コードを返すタスク / A task that returns the start result code.

StartExperimentAsync(string?)

計測を開始し、必要に応じて **csvOutputFolder** への CSV ログ出力も開始する。装置が計測開始を通知した時点で完了する / Starts measurement. Optionally begins CSV logging to **csvOutputFolder**. Completes when the device confirms measurement has started.

```
public Task<EnmSdkResult> StartExperimentAsync(string? csvOutputFolder = null)
```

Parameters

csvOutputFolder [string](#)

CSV 出力先フォルダ（null の場合は出力しない。空文字列は現在の作業ディレクトリ） / CSV output folder path (no output when null; an empty string uses the current working directory).

Returns

[Task](#) <[EnmSdkResult](#)>

開始処理の結果コードを返すタスク / A task that returns the start result code.

StartReferencePeriod(string)

SMD 計算に使用する参照期間の開始を記録する。同時に開ける参照期間は 1 つのみで、新しい開始は前回設定を上書きする / Marks the start of a reference period used for SMD calculation. Only one reference period can be open at a time; starting a new one overwrites the previous.

```
public EnmSdkResult StartReferencePeriod(string name)
```

Parameters

name [string](#)

参照期間名 / Reference period name.

Returns

[EnmSdkResult](#)

結果コード / Result code.

Delegate

Experiment.NotifyInstantFrameCallback

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

SMD 値付きリアルタイム計測フレームごとに呼び出されるコールバック / Callback invoked for each real-time measurement frame with SMD values.

```
public delegate void Experiment.NotifyInstantFrameCallback(InstantFrame frame)
```

Parameters

frame [InstantFrame](#)

通知されたリアルタイムフレーム / The notified real-time frame.

Delegate Experiment.NotifyStageStartedCallback

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

ブロックの各フェーズ開始時に呼び出されるコールバック / Callback invoked when each phase of a block begins.

```
public delegate void Experiment.NotifyStageStartedCallback(BlockStage stage)
```

Parameters

stage [BlockStage](#)

開始したフェーズ / The phase that just began.

Class FrameData

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

完了した Block に格納される単一の処理済み計測フレーム / A single processed measurement frame stored in a completed Block.

```
public sealed class FrameData : InstantFrame
```

Inheritance

[object](#)  ← [InstantFrame](#) ← FrameData

Inherited Members

[InstantFrame.Raw](#) , [InstantFrame.SmdBrainIndex_left](#) , [InstantFrame.SmdBrainIndex_right](#)

Properties

dTHbL_leftSD3_1cm_Processed

dTHbL(leftSD3-1cm)Processed — 左側の基線補正済み差分 (L3-L1) / Baseline-corrected difference (L3-L1) on the left

```
[JsonPropertyName("dTHbL(leftSD3-1cm)Processed")]  
public double dTHbL_leftSD3_1cm_Processed { get; set; }
```

Property Value

[double](#) 

dTHbL_rightSD3_1cm_Processed

dTHbL(rightSD3-1cm)Processed — 右側の基線補正済み差分 (R3-R1) / Baseline-corrected difference (R3-R1) on the right

```
[JsonPropertyName("dTHbL(rightSD3-1cm)Processed")]
```

```
public double dTHbL_rightSD3_1cm_Processed { get; set; }
```

Property Value

[double](#)

Class InstantFrame

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

SMD 値を含むハイレベルのリアルタイム計測フレーム / High-level real-time measurement frame including SMD values.

```
public class InstantFrame
```

Inheritance

[object](#)  ← InstantFrame

Derived

[FrameData](#)

Properties

Raw

このフレームの生計測データ / Raw measurement data for this frame

```
public ExBrainMeasureData Raw { get; set; }
```

Property Value

[ExBrainMeasureData](#)

SmdBrainIndex_left

SmdBrainIndex(left). 参照期間未設定またはウィンドウ未充足時は null / Null if reference period not set or window not filled

```
[JsonPropertyName("SmdBrainIndex(left))"]  
public double? SmdBrainIndex_left { get; set; }
```

Property Value

[double](#)?

SmdBrainIndex_right

SmdBrainIndex(right). 参照期間未設定またはウィンドウ未充足時は null / Null if reference period not set or window not filled

```
[JsonPropertyName("SmdBrainIndex(right))"]  
public double? SmdBrainIndex_right { get; set; }
```

Property Value

[double](#)?

Class JsonExtensions

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

JSON シリアライズ拡張メソッド群 / Extension methods for JSON serialization.

```
public static class JsonExtensions
```

Inheritance

[object](#)  ← JsonExtensions

Methods

ToJson(Block)

Block を JSON 文字列へシリアライズする / Serializes the Block to a JSON string.

```
public static string ToJson(this Block block)
```

Parameters

block [Block](#)

シリアライズ対象の Block / The Block to serialize.

Returns

[string](#) 

Block の JSON 文字列表現 / A JSON string representation of the Block.

Class MetaData

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

計測ブロックの時間フェーズ設定。所有者は API / Configuration for a measurement block's timing phases.
Owned by the API.

```
public sealed class MetaData
```

Inheritance

[object](#)  ← MetaData

Constructors

MetaData(ExBrainApi, string, int, int, int, int)

計測ブロックのフェーズ時間定義を作成する / Creates phase-duration metadata for a measurement block.

```
public MetaData(ExBrainApi api, string name, int preFittingSeconds, int taskSeconds, int  
calmDownSeconds, int postFittingSeconds)
```

Parameters

api [ExBrainApi](#)

この MetaData を保持する API インスタンス / The API instance that will own this MetaData.

name [string](#) 

ブロック種別名（例: タスク名） / Block type name (e.g. task name).

preFittingSeconds [int](#) 

タスク前の事前装着調整フェーズ時間（秒） / Duration of the pre-fitting phase before the task.

taskSeconds [int](#) 

タスクフェーズ時間（秒、負値で手動制御） / Duration of the task phase (use negative to control timing manually).

`calmDownSeconds` [int](#)

タスク後のクールダウンフェーズ時間（秒） / Duration of the calm-down phase after the task.

`postFittingSeconds` [int](#)

クールダウン後の事後装着調整フェーズ時間（秒） / Duration of the post-fitting phase after calm-down.

Exceptions

`InvalidOperationException`

ネイティブ MetaData の作成に失敗した場合 / Thrown when native metadata creation fails.

Enum StatisticalTest

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

TaskComparison で使用される統計検定種別 / Statistical test used in a TaskComparison.

```
public enum StatisticalTest
```

Fields

MannWhitneyU = 4

Mann-Whitney U 検定: 対応なし・ノンパラメトリック / Mann-Whitney U test: unpaired, non-parametric.

None = 0

検定未実施 (例: 片側が無効) / No test was performed (e.g. the side was disabled).

PairedT = 3

対応あり t 検定: 対応あり・パラメトリック / Paired t-test: paired, parametric.

StudentT = 2

Student の t 検定: 対応なし・パラメトリック・等分散を仮定 / Student's t-test: unpaired, parametric, equal variances assumed.

WelchT = 1

Welch の t 検定: 対応なし・パラメトリック・等分散を仮定しない / Welch's t-test: unpaired, parametric, unequal variances assumed.

WilcoxonSignedRank = 5

Wilcoxon 符号付順位検定: 対応あり・ノンパラメトリック / Wilcoxon signed-rank test: paired, non-parametric.

Enum Tail

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

統計検定の片側・両側指定 / Tail direction for statistical tests.

```
public enum Tail
```

Fields

Left = 1

左片側検定: グループ A がグループ B より有意に小さいかを検出 / Left-tailed test: detects whether group A is significantly less than group B.

Right = 2

右片側検定: グループ A がグループ B より有意に大きいかを検出 / Right-tailed test: detects whether group A is significantly greater than group B.

TwoSided = 0

両側検定: どちら方向の差も検出 / Two-sided test: detects differences in either direction.

Class TaskComparison

Namespace: [ExBrainSdk.HighLevel](#)

Assembly: ExBrainSdk.dll

2つのブロック集合間の統計比較結果。インスタンス生成は static メソッドを使用 / Result of a statistical comparison between two sets of blocks. Use the static methods to create instances.

```
public sealed class TaskComparison
```

Inheritance

[object](#)  ← TaskComparison

Properties

dTHbL_left_difference

左側比較の差分。パラメトリック時は meanA-meanB、ノンパラ時は medianA-medianB / meanA-meanB (parametric) or medianA-medianB (non-parametric) for the left-side comparison.

```
public double dTHbL_left_difference { get; }
```

Property Value

[double](#) 

dTHbL_left_pValue

dTHbL(leftSD3-1cm)Processed 比較の p 値 / P-value for the left-side dTHbL(leftSD3-1cm)Processed comparison.

```
public double dTHbL_left_pValue { get; }
```

Property Value

[double](#) 

dTHbL_left_testKind

dTHbL(leftSD3-1cm)Processed 比較に使用された検定種別 / The statistical test used for the left-side dTHbL(leftSD3-1cm)Processed comparison.

```
public StatisticalTest dTHbL_left_testKind { get; }
```

Property Value

[StatisticalTest](#)

dTHbL_right_difference

右側比較の差分。パラメトリック時は meanA-meanB、ノンパラ時は medianA-medianB / meanA-meanB (parametric) or medianA-medianB (non-parametric) for the right-side comparison.

```
public double dTHbL_right_difference { get; }
```

Property Value

[double](#)

dTHbL_right_pValue

dTHbL(rightSD3-1cm)Processed 比較の p 値。右側無効時は NaN / P-value for the right-side dTHbL(rightSD3-1cm)Processed comparison. NaN when right side is disabled.

```
public double dTHbL_right_pValue { get; }
```

Property Value

[double](#)

dTHbL_right_testKind

dTHbL(rightSD3-1cm)Processed 比較に使用された検定種別。右側無効時は None と NaN / The statistical test used for the right-side dTHbL(rightSD3-1cm)Processed comparison. None and NaN p-value when right side is disabled.

```
public StatisticalTest dTHbL_right_testKind { get; }
```

Property Value

[StatisticalTest](#)

Methods

CompareBetweenGroupsTask(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, bool?, Tail)

独立した 2 群の参加者間で同一タスクを比較する / Compares a task between two independent groups of participants.

```
public static TaskComparison CompareBetweenGroupsTask(ExBrainApi api, IReadOnlyList<Block> blocksA, IReadOnlyList<Block> blocksB, bool? isParametric = null, bool? isEqualVariance = null, Tail tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocksA [IReadOnlyList](#) <[Block](#)>

グループ A の参加者ブロック群。blocksB と参加者 ID が重複しないこと / Blocks from participants in group A. Participant identities must not overlap with blocksB.

blocksB [IReadOnlyList](#) <[Block](#)>

グループ B の参加者ブロック群 / Blocks from participants in group B.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

isEqualVariance [bool](#)?

null: Levene 検定で自動判定、true: 等分散、false: 不等分散（isParametric=true の場合のみ使用） / null = auto-detect via Levene's test, true = equal variance, false = unequal. Only used when isParametric is true.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

InvalidOperationException

比較処理に失敗した場合 / Thrown when comparison processing fails.

CompareBetweenParticipantsTasks(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, bool?, Tail)

2つの参加者集合間でタスクを比較する（[CompareBetweenGroupsTask\(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, bool?, Tail\)](#) の別名） / Compares a task between two sets of participants (alias for [CompareBetweenGroupsTask\(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, bool?, Tail\)](#)).

```
public static TaskComparison CompareBetweenParticipantsTasks(ExBrainApi api,
    IReadOnlyList<Block> blocksA, IReadOnlyList<Block> blocksB, bool? isParametric = null, bool?
    isEqualVariance = null, Tail tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocksA [IReadOnlyList](#) <[Block](#)>

グループ A の参加者ブロック群。blocksB と参加者 ID が重複しないこと / Blocks from participants in group A. Participant identities must not overlap with blocksB.

blocksB [IReadOnlyList](#) <[Block](#)>

グループ B の参加者ブロック群 / Blocks from participants in group B.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

isEqualVariance [bool](#)?

null: Levene 検定で自動判定、true: 等分散、false: 不等分散（isParametric=true の場合のみ使用） / null = auto-detect via Levene's test, true = equal variance, false = unequal. Only used when isParametric is true.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

[InvalidOperationException](#)

比較処理に失敗した場合 / Thrown when comparison processing fails.

ComparePairedGroupTasks(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, Tail)

複数参加者にまたがって 2 つのタスクを対応付け比較する / Compares two tasks across multiple participants, pairing each participant's block in A with their block in B.

```
public static TaskComparison ComparePairedGroupTasks(ExBrainApi api, IReadOnlyList<Block>
blocksA, IReadOnlyList<Block> blocksB, bool? isParametric = null, Tail tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocksA [IReadOnlyList](#) <[Block](#)>

タスク A のブロック群（参加者ごとに 1 つ）。index で blocksB と対応し、参加者 ID が一致する必要あり /
Blocks for task A, one per participant. Paired with blocksB by index; participant identities must match.

blocksB [IReadOnlyList](#) <[Block](#)>

タスク B のブロック群。index で blocksA と対応 / Blocks for task B, paired with blocksA by index.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

[InvalidOperationException](#)

比較処理に失敗した場合 / Thrown when comparison processing fails.

[CompareTaskAndBaseline\(ExBrainApi, IReadOnlyList<Block>, bool?, Tail\)](#)

同一参加者ブロック内でタスク期間と事前装着調整（ベースライン）期間を比較する / Compares the task period against the pre-fitting (baseline) period within the same participant's blocks.

```
public static TaskComparison CompareTaskAndBaseline(ExBrainApi api, IReadOnlyList<Block>
blocks, bool? isParametric = null, Tail tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocks [IReadOnlyList](#) [<Block>](#)

比較対象ブロック群。すべて同一参加者に属する必要あり / Blocks to compare. All must belong to the same participant.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

[InvalidOperationException](#)

比較処理に失敗した場合 / Thrown when comparison processing fails.

`CompareWithinParticipantPrePost(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool, bool?, bool?, Tail)`

同一参加者の事前条件と事後条件のブロックを比較する（例: 介入前後） / Compares pre- and post-condition blocks for the same participant (e.g. before/after intervention).

```
public static TaskComparison CompareWithinParticipantPrePost(ExBrainApi api,
    IReadOnlyList<Block> blocksA, IReadOnlyList<Block> blocksB, bool isPaired = true, bool?
    isParametric = null, bool? isEqualVariance = null, Tail tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocksA [IReadOnlyList](#) <[Block](#)>

事前条件のブロック群。すべて同一参加者に属する必要あり / Blocks for the pre condition. All must belong to the same participant.

blocksB [IReadOnlyList](#) <[Block](#)>

事後条件のブロック群。すべて同一参加者に属する必要あり / Blocks for the post condition. All must belong to the same participant.

isPaired [bool](#)

true: 対応あり比較（repeatIndex で対応付け）、false: 対応なし / true = paired comparison (blocks matched by repeatIndex); false = unpaired.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

isEqualVariance [bool](#)?

null: Levene 検定で自動判定、true: 等分散、false: 不等分散（isPaired=false かつ isParametric=true の場合のみ使用） / null = auto-detect via Levene's test, true = equal variance, false = unequal. Only used when isPaired is false and isParametric is true.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

InvalidOperationException

比較処理に失敗した場合 / Thrown when comparison processing fails.

CompareWithinParticipantTasks(ExBrainApi, IReadOnlyList<Block>, IReadOnlyList<Block>, bool?, Tail)

同一参加者の繰り返しブロックで 2 つのタスクを比較する / Compares two tasks performed by the same participant across repeated blocks.

```
public static TaskComparison CompareWithinParticipantTasks(ExBrainApi api,
    IReadOnlyList<Block> blocksA, IReadOnlyList<Block> blocksB, bool? isParametric = null, Tail
    tail = Tail.TwoSided)
```

Parameters

api [ExBrainApi](#)

ログ出力に使用する API インスタンス / The API instance used for logging.

blocksA [IReadOnlyList](#) <[Block](#)>

タスク A のブロック群（繰り返しごとに 1 つ）。同一参加者 ID である必要あり / Blocks for task A, one per repeat. All must share the same participant identity.

blocksB [IReadOnlyList](#) <[Block](#)>

タスク B のブロック群。index と repeatIndex で blocksA と対応づけ / Blocks for task B, paired with blocksA by index and repeatIndex.

isParametric [bool](#)?

null: Shapiro-Wilk で自動判定、true: パラメトリック、false: ノンパラメトリック / null = auto-detect via Shapiro-Wilk, true = parametric, false = non-parametric.

tail [Tail](#)

仮説検定の片側・両側指定 / Tail direction for the hypothesis test.

Returns

[TaskComparison](#)

比較結果 / Comparison result.

Exceptions

InvalidOperationException

比較処理に失敗した場合 / Thrown when comparison processing fails.